

ContextSniper: AntTrail’s Token-Efficient Code Memory for Repository-Level Program Repair

Chiwang Luk^{1,†} Matin Mohammad Najafi¹ Zhifeng Jia¹
 Wei Yang¹ Xiuchang Li¹ Jinwei Zhu¹
 Yang Ren¹ Lei Chen² Gao Cong^{3,†}

¹Huawei ²HKUST(GZ) ³Nanyang Technological University

Abstract

Large language model agents can repair real repository issues, but they often spend large context budgets on whole-file reads, broad searches, and long terminal outputs where useful evidence is mixed with irrelevant code and logs. This paper presents ContextSniper, AntTrail’s code-repair module for precision evidence selection in repository-level program repair, part of AntTrail’s broader agent-memory engine¹. ContextSniper indexes code and action memory as three views (compact abstract, structured overview, full content), retrieves candidates with a hybrid ranker that fuses semantic embeddings, BM25, symbolic (ctags-style) metadata, and graph relations, filters long tool output through an intention-aware context gate, and returns compact evidence packets while keeping full source recoverable on demand. In a matched 50-task-per-condition comparison on SWE-bench Lite (same tasks, baseline vs. ContextSniper), ContextSniper reduces total token use by 51.5% and logged cost by 36.4% for OpenClaw, with a comparable submitted-resolution rate (a difference of one task per 50). In a separate five-task comparison, ContextSniper beats existing memory- and RAG-style integrations (mem0, Letta/MemGPT, Serena, LlamaIndex-style RAG) on token efficiency. These results suggest a dedicated context-access layer can substantially cut token and cost overhead for repository-level repair agents without a measurable loss in repair quality.²

Keywords: Repository-Level Program Repair, Coding Agents, Code Memory, Agentic Memory, Retrieval-Augmented Software Engineering, Token Efficiency

1 Introduction

1.1 Motivation

When a coding agent cannot find the evidence it needs, it does not stop; it rereads. A failed patch attempt on a real repository issue often triggers another whole-file read, another broad search, another look at a stack trace already seen once, producing a read–try–fail–reread loop that spends tokens without necessarily finding the missing evidence [1, 2]. This failure mode matters because large language model code agents are now evaluated and deployed for exactly this kind of repository-level work: repair benchmarks and agent systems require models to inspect codebases, execute tools, and generate source-code patches rather than only complete isolated snippets [1, 3–5]. Production coding agents apply the same inspect-edit-test loop to real development and debugging work. In these workflows, context is not merely auxiliary text: it is the agent’s working record of repository state. Losing a single file path, failing-test trace, API precondition, or symbol relationship can derail localization and cause an otherwise plausible patch to target the wrong code, which mirrors recent long-horizon agent work arguing that context must preserve precise actions, observations, and evolving state rather than simply maximize length [6].

The difficulty is that prevailing agents obtain this evidence through raw repository access. As illustrated in Figure 1A, a task prompt is mixed with whole-file reads, broad search results, and long terminal logs. As interactions accumulate, the active context grows with both useful evidence and stale exploration residue. This growth creates two coupled bottlenecks for repository repair. First, it raises inference cost and latency because every later reasoning step must carry more tokens. Second, it degrades decision quality because the few lines that explain the failure are diluted by unrelated helpers, repeated stack traces, build noise, and earlier failed attempts. Long-context studies show that models may

[†]Corresponding authors.

¹AntTrail: gitcode.com/datagallery/AntTrail.

²Evaluation harness for this study: github.com/Calluking/ContextSniper.

fail to use relevant information reliably when it is buried inside long inputs [7]; repository repair makes this problem concrete because the evidence needed for a patch is often surrounded by unrelated code and runtime output.

Naive context reduction is not enough. Simple truncation can remove the only test assertion or import edge that explains a bug, while generic summarization can blur provenance, line numbers, and editability. Effective context management for coding agents must reduce aggressively while retaining heterogeneous repair signals: issue intent, code structure, symbol definitions, failing observations, command outcomes, and the updated state after edits. Retrieval-augmented generation and repository-memory systems motivate a separation between knowledge storage and prompt construction [2, 8, 9], but repository repair also requires control over what read and command output is allowed to enter the active context in the first place.

This paper builds on a different view of context: evidence, not ballast. AntTrail is a general agent-memory engine for durable task context, designed to store, index, retrieve, and budget the information an agent may need instead of leaving it buried in raw transcripts, whole artifacts, or unfiltered tool output [10]. ContextSniper is AntTrail’s module specialized for repository-level program repair. Figure 1B shows the resulting workflow. Instead of passing raw repository and runtime output directly to the LLM, ContextSniper first treats it as candidate evidence. We describe this evidence-capture process as *sniping*: the system keeps task-relevant code and runtime signals, removes surrounding low-value regions, and assembles the surviving material into a compact evidence packet for the agent.

The intended operating point is shown in Figure 1C. More context can help when it adds missing evidence, but after the agent has enough information, additional files and logs mostly add cost and distraction. ContextSniper therefore does not try to maximize context size or compress everything uniformly. It tries to keep the agent near the useful-evidence region: enough code and runtime evidence to localize and repair the issue, but little enough surrounding noise to reduce tokens, turns, and confusion. Rather than replacing the host coding agent, ContextSniper targets the context-access layer between the agent and the repository. It maintains local code memory, retrieves compact task-relevant code, suppresses noisy read and command outputs, and synchronizes memory with repository edits, so the prompt carries the current evidence packet rather than the entire exploration history.

1.2 Problem Statement

Repository-level program repair requires an agent to connect an issue description to the relevant implementation, tests, interfaces, and execution behavior of a codebase [3]. This is difficult because the necessary evidence is rarely isolated in one function. It may be distributed across multiple files, imported dependencies, class hierarchies, configuration logic, and failing tests. Before an agent can generate a correct patch, it must first localize the code that matters, a bottleneck also targeted by localization and repository-graph methods [5, 11, 12].

Existing agents usually acquire this context through raw repository tools. They read files directly, run search commands, inspect command outputs, and then decide what to read next [1, 4]. This workflow is general, but it treats context accumulation as the default path to evidence. It provides little memory across repeated exploration steps, little control over the amount of irrelevant output entering the context window, and limited support for keeping retrieved context aligned with files that change during the repair process. Recent codebase-memory and graph-based systems address parts of this problem by giving agents structured repository access, but context selection and runtime output control remain important practical concerns [2, 9].

This paper studies the following problem: how can existing repository-level code agents transform raw repository and runtime context into compact repair evidence while leaving the agent’s core reasoning and editing loop intact? The problem has two coupled objectives. The first is to reduce waste: fewer irrelevant tokens, fewer repeated exploration turns, lower cost, and shorter latency. The second is to reduce confusion: cleaner evidence packets that preserve code localization, patch reasoning, and validation quality while removing surrounding noise.

ContextSniper connects to three lines of prior work that target the same context bottleneck from different angles: agent tools and agent-computer interfaces for repository-level software engineering, prompt and retrieval compression, and output-side filtering for agent tools. Agent-tool work shapes how models navigate, search, and edit repositories, but the active context it produces remains tightly coupled to raw exploration output. Prompt- and retrieval-compression methods operate on text or chunks already selected for a prompt, after the selection decision has been made. Output-filtering systems trim noisy tool output but do not maintain a synchronized code memory across a repair run. ContextSniper is distinguished from all three: it gates raw repository and runtime output at the point where it would otherwise enter the model’s context, while keeping that gating synchronized with a task-time code memory, rather than compressing an already-assembled prompt or replacing the host agent’s tool loop. We position ContextSniper against specific systems in each of these three lines in Section 2.



Figure 1. ContextSniper turns noisy repository context into compact repair evidence. (A) and (B) share the same host-agent workflow; only the context-acquisition module differs. (A) Native `Grep`, `Read`, and `Bash` outputs accumulate into an unfiltered prompt context, and failed attempts reread similar files. (B) A ContextSniper insertion layer replaces broad discovery with `search_code` excerpts and gates long `Read/Bash` output before the same host agent assembles a compact prompt. (C) A conceptual illustration, not measured data, of the intended operating point: context acquisition passes through four stages, instruction, retrieval and edits, sufficient evidence, and context flooding, with ContextSniper reaching the operating point early and stopping while raw `grep/ls` access keeps spending tokens through redundant reads and only loses accuracy once context floods. The bars give the measured average token use per task (Table 3).

Table 1. Positioning ContextSniper relative to repository-level repair systems.

Family	Examples	Primary optimization target	Context-access boundary
Benchmark setting	SWE-bench [3]	Real issue-to-patch tasks over full repositories.	Defines the task and validation setting, but does not prescribe how evidence should be retrieved or bounded.
Autonomous repair agents	SWE-agent and AutoCodeRover [1, 4]	Tool use, repository navigation, search, edit, test, and agent-computer interface design.	Context selection is largely coupled to the host agent’s exploratory tool loop and the raw outputs that loop observes.
Pipeline-style repair	Agentless [5]	A simpler decomposition into localization, repair, and validation.	Localization is a pipeline stage; reusable task-time memory, runtime-output gating, and edit synchronization are not the main abstraction.
Context-access layer	ContextSniper (AntTrail)	Evidence retrieval, budgeting, filtering, and synchronization for an existing host agent.	The repair agent remains responsible for reasoning and editing; the context layer controls which repository and runtime evidence enters the prompt.

1.3 Contributions

We make the following contributions:

- We introduce ContextSniper, AntTrail’s code-repair module. It sits between a host agent and a target repository, converting noisy repository access into compact evidence packets while leaving the agent’s core reasoning and editing loop intact.
- We propose a two-family memory hierarchy backed by AGFS (Agent File System), in which code memory and action memory are each indexed as three views: L0 compact abstract, L1 structured overview, and L2 full content.
- We develop adaptive top- k retrieval that fuses semantic embeddings, BM25 lexical scoring, ctags-style symbolic metadata (symbol names, kinds, and locations), and graph relations through weighted reciprocal rank fusion (RRF), with a ripgrep-based text-search fallback [13] when the index misses a query.
- We add an intention-aware context gate and a two-phase memory-repository synchronization that snipe noisy tool output and keep local code memory aligned with the working tree after agent edits.
- We evaluate ContextSniper on SWE-bench Lite with OpenClaw as the host agent, comparing its baseline against ContextSniper in terms of token usage, tool behavior, and repair outcomes [3, 14]. ContextSniper reduces total token use by 51.5%, with a comparable submitted-resolution rate.

2 Background and Related Work

2.1 Repository-Level Program Repair

Repository-level program repair asks a model or agent to modify an existing codebase in response to a natural-language issue report, rather than solving an isolated programming exercise. SWE-bench made this setting a standard evaluation target by collecting real GitHub issues and pull requests from Python repositories; its tasks require models to inspect a repository, understand issue-specific behavior, and coordinate edits across files, tests, and execution environments [3]. This benchmark shifted automated repair from single-function generation toward interactive repository work, where finding the right evidence can be as important as generating the final patch.

Recent systems can be separated along four dimensions: how autonomous the model is, how localization is performed, whether memory persists across interactions or tasks, and whether raw repository or runtime output is filtered before entering the prompt. SWE-agent studies the agent-computer interface, showing that repository navigation, file editing, and test execution tools can strongly affect software-engineering agent behavior [1]. AutoCodeRover uses a more software-engineering-oriented autonomous workflow, combining LLM reasoning with structured code search and test-aware fault localization [4]. Agentless, by contrast, separates localization, repair, and patch validation into a simpler pipeline rather than giving the model a fully autonomous tool loop [5]. Table 1 summarizes the resulting boundary.

This distinction is central to the paper’s contribution. ContextSniper is not a new standalone repair agent and does not replace the host model’s reasoning loop. Instead, it targets the context-access layer used by repair agents:

how repository evidence is retrieved, bounded, selected, filtered, and kept synchronized with edits. This makes it complementary to autonomous systems such as SWE-agent and AutoCodeRover and pipeline-style systems such as Agentless.

2.2 Code Memory and Agentic Memory

Retrieval-augmented generation introduced a general pattern for combining a parametric language model with explicit non-parametric memory, allowing retrieved passages to condition generation without retraining the model [8]. Software-engineering agents extend this idea because useful knowledge may include source code, documentation, prior fixes, execution traces, project conventions, and failure histories. The important distinction for this paper is not only whether memory exists, but what scope the memory covers and when it is updated.

One line of work stores cross-task repair experience. SWE-Exp distills concise knowledge from prior agent trajectories into an experience bank for issue resolution [15]. ExpeRepair uses episodic and semantic memories to retrieve concrete repair demonstrations and higher-level repair insights [16]. MemGovern expands the source of experience to governed human issue-tracking data, aiming to move agents beyond local context alone [17]. A second line studies memory granularity: structurally aligned subtask-level memory argues that whole-instance memories can be too coarse and stores or retrieves experience at the level of functional subtasks [18]. A third line studies domain adaptation, as in MEMCoder, which evolves memory for private-library-oriented code generation where the model must learn internal API usage patterns and constraints over time [19].

These systems show that memory is becoming central to software agents, but their main emphasis is experience reuse across tasks, histories, or domains. ContextSniper focuses on task-time local code memory. Its memory is built from the current repository snapshot, queried during the current repair run, refreshed as files change, and used to control not only code retrieval but also what read, search, and shell outputs enter the active context. This narrower scope is intentional: it addresses repeated exploration and noisy context within a single issue-resolution process, while leaving room for future integration with cross-task experience memory.

2.3 Repository Structure, Localization, and Context Management

Repository-level repair depends on localization before generation. If the agent reads the wrong files or misses the right symbol, later patch reasoning is likely to fail regardless of model size. Prior work therefore uses repository structure, history, or graphs to narrow the search space. RepoGraph provides a repository-level code graph as a plug-in module for AI software-engineering systems [9]. KGCompass uses repository-aware knowledge graphs over artifacts such as issues, pull requests, files, classes, and functions to guide fault localization and repair [11]. Repository-memory localization work builds non-parametric memory from historical commits and linked issues, showing that a project’s change history can help agents identify likely fix locations [12]. Learning to Commit similarly studies online repository memory for generating pull requests that respect project-specific conventions and architectural patterns [20].

Persistent codebase memory systems address a closely related practical bottleneck: LLM coding agents often explore repositories through repeated file reads and broad search commands. Codebase-Memory constructs a Tree-Sitter-based knowledge graph exposed through the Model Context Protocol and reports large token reductions for codebase exploration tasks [2]. This line of work supports the central premise of ContextSniper: repository agents need structured access to code, not only larger context windows. ContextSniper differs by treating the final prompt as a budgeted evidence packet that must preserve source provenance, filter runtime noise, and remain synchronized with the repository after edits.

Long-context models do not remove this selection problem. Empirical work on long-context language models shows that performance can degrade when relevant information is buried in the middle of long inputs, even for models designed to accept long contexts [7]. This result is not a direct evaluation of repository repair, but it motivates caution: adding more files, logs, and command output is not equivalent to making repair evidence easier to use. The gap left by prior work is therefore a context-management problem with four requirements: select task-relevant local repository evidence under a budget, preserve provenance so the host agent can inspect and edit the source, suppress low-value runtime output without losing diagnostic anchors, and keep retrieved memory aligned with changes made during the repair run.

2.4 Prompt and Retrieval Compression

A separate line of work optimizes prompt and retrieval compression once content has already been selected for a prompt. LLMLINGUA and LONGLLMLINGUA drop low-information tokens via a small LM, achieving up to $20\times$ compression with small quality loss and explicit mitigation of the lost-in-the-middle failure mode [21, 22]. RECOMP

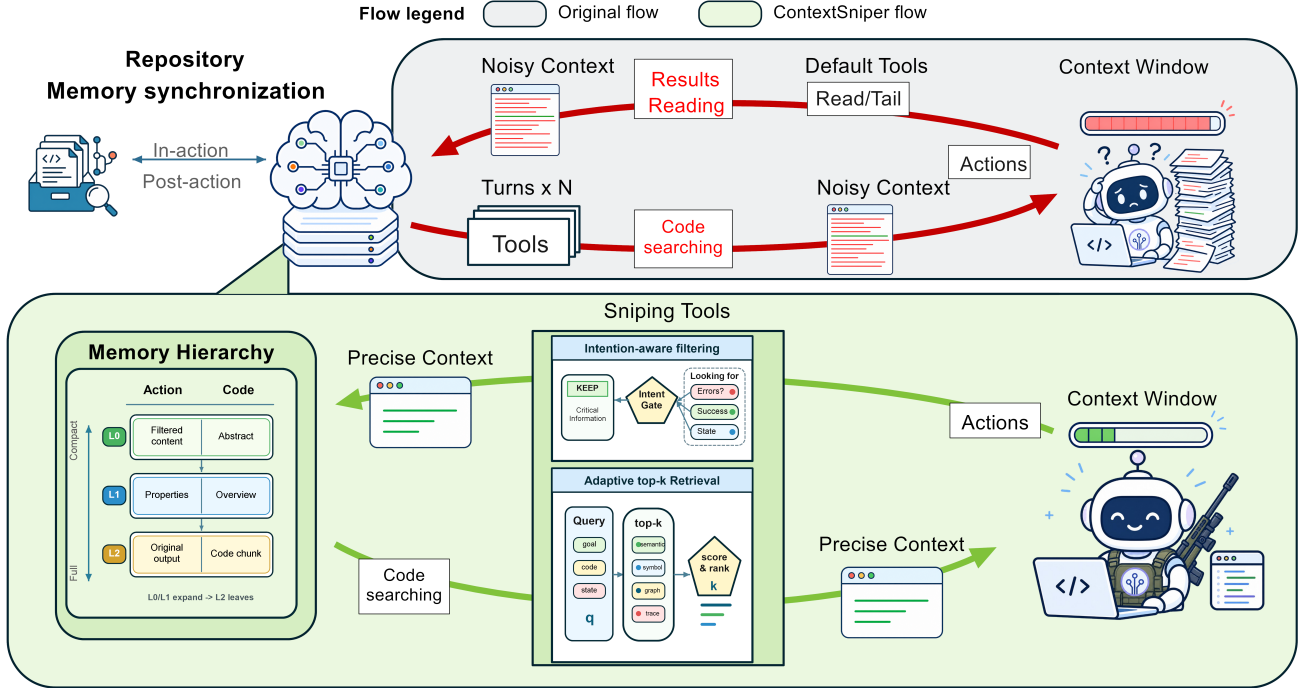


Figure 2. ContextSniper architecture. The upper path shows the baseline interaction loop, where broad search, long file reads, repeated actions, and raw command output accumulate noisy context before reaching the model’s context window. The lower path shows the ContextSniper insertion layer. Repository and action evidence are stored in an AGFS-backed memory hierarchy, synchronized with the current working tree, retrieved through hybrid code search, and sniped by intention-aware context gates before compact evidence returns to the same host-agent loop.

inserts a learned compressor between retrieval and the LM and ships both extractive and abstractive variants for retrieval-augmented generation (RAG) settings [23]. COMPACT makes the compression rate query-dependent, deciding per retrieved document how aggressively to shorten it [24]. SELECTIVE CONTEXT filters tokens by self-information [25], while AUTOCOMPRESSOR trains the model to emit summary vectors that compress earlier context into the KV cache [26]. Open-source engineering tools in this layer include CTXBUDGETER, which compiles, audits, governs, and visualizes context packs before model calls with token budgets, policy checks, provenance, and cache planning [27]. These methods operate on text or retrieved chunks already selected for a prompt; ContextSniper operates one layer earlier for repository repair, deciding which tool outputs and code excerpts are allowed to enter that text at all.

2.5 Agent-Tool Output Filtering

A third line filters agent-tool outputs before they enter the model context. RTK rewrites shell commands through a Rust CLI proxy and reports 60–90% token savings on common development commands by filtering, grouping, truncating, and deduplicating outputs [28]. HEADROOM generalizes this idea beyond shell commands to tool outputs, logs, files, RAG chunks, and conversation history through a library, proxy, and MCP server, reporting 60–95% fewer tokens on real agent workloads while retaining local-first operation and reversible retrieval of originals [29]. BEARING addresses context growth at the workflow level: it separates planning from execution, runs tasks in fresh agent processes, and passes forward task summaries and relevant-file hints rather than an entire accumulated conversation [30]. ContextSniper is closest to this family, but specializes the output gate for repository-level repair: it combines tool-output filtering with synchronized local code memory, so the host agent receives compact, task-relevant repair evidence without changing its outer workflow. Existing agent tools, prompt compressors, and output filters each improve one link of the evidence chain; to our knowledge, none of them gates raw repository and runtime output for repair while maintaining synchronized code memory. ContextSniper combines these ideas into a single context-access layer and is evaluated against the prevailing raw-access workflow on a repository-level repair benchmark [3], rather than against an abstract compression ratio on a synthetic long-context task.

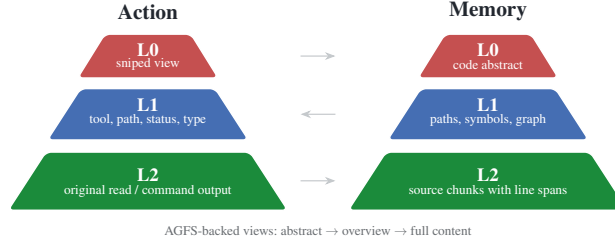


Figure 3. Two-family L0-L2 memory hierarchy. Action memory stores compact action views, action metadata, and recoverable original tool output. Code memory stores code abstracts, structured routing metadata, and source-grounded chunks.

3 ContextSniper: AntTrail’s Code-Oriented Memory Module

3.1 System Overview

Within AntTrail, ContextSniper is the module specialized for repository-level program repair. It is inserted between a host coding agent and a target repository, but it does not replace the agent’s planning loop, editor, shell, or validation procedure. Instead, it changes the evidence path. In the baseline path in Figure 2, the agent explores the repository through broad search, whole-file reads, and command outputs. These outputs often contain the needed repair evidence, but they also carry unrelated functions, repeated logs, and stale exploratory context into the model window. ContextSniper keeps the same outer workflow while changing what the model receives from each interaction.

The core principle is to retrieve broadly but expose narrowly. ContextSniper stores full repository and action evidence outside the prompt, searches that evidence with multiple retrieval signals, and returns only the snippets and runtime facts that are likely to support localization, patch construction, or validation. This is different from generic compression: the system is not trying to summarize everything the agent saw. It is trying to remove unwanted context while preserving source-grounded evidence. Every returned code excerpt keeps its repository path and line region, and every retained runtime signal is tied to the tool call, command, or edit that produced it.

After this overview, the architecture has five implementation components. Section 3.2 describes the AGFS-backed memory hierarchy used to store code and action evidence. Section 3.3 describes synchronization between memory and the working repository. Section 3.4 describes code search and adaptive top- k retrieval, including semantic embeddings, BM25 lexical scoring, symbol ranking, graph evidence, reciprocal-rank fusion, and ripgrep fallback [13, 31, 32]. Section 3.5 describes the context gate that snipes, or removes low-value regions from, long reads and command outputs before they enter the model prompt. Section 3.6 defines the host-agent boundary; the concrete pilot adapters are described in Section 4.1.

3.2 Memory Hierarchy

For code repair, the memory layer is organized into two families: code memory for repository evidence and action memory for evidence produced during the repair run. Both are stored outside the prompt in an AGFS-backed local memory service [33]. AGFS (Agent File System) stores each memory node as a directory with three text views: an abstract, an overview, and full content. ContextSniper indexes these views as three retrieval levels: L0, L1, and L2.

L0 is the compact abstract level: a short model-facing view used for broad recall and, for long reads or shell output, as the returned sniped view. L1 is the structured overview level, storing routing and ranking metadata such as paths, line ranges, symbols, signatures, ctags-style symbol kinds, BM25 lexical documents, graph relations, tool names, command status, and output type [34]. L2 is the recoverable full-content level. For code memory, L2 contains source-grounded chunks with path and line headers; Python files use an abstract syntax tree (AST) chunker for top-level classes, functions, and async functions, while other files fall back to overlapping line windows. For action memory, L2 stores the original unfiltered read or command output. Thus L0 gives compact recall, L1 gives structured routing, and L2 preserves editable or recoverable source evidence.

3.3 Memory-Repository Synchronization

Memory-repository synchronization keeps code memory aligned with the current working tree. A repair run starts from a checked-out repository, but the repository does not stay fixed: the agent edits files, executes tests, reads new regions, and may trigger generated artifacts or logs. If retrieval returns an old version of a function after an edit, the compact context becomes actively harmful. ContextSniper therefore treats memory freshness as part of source grounding.

Synchronization happens in two phases. The first phase runs after agent actions through runtime hooks. When the host agent searches code, reads a file, edits source, or runs a command, ContextSniper records touched paths, returned

Input: query q , scope S , memory M , state R , action memory A , ceiling k_0 , budget B .
Output: ranked L2 evidence packet E .

```

function SEARCHCODE( $q, S, M, R, A, k_0, B$ )
   $intent \leftarrow \text{INFERINTENT}(q, A)$ 
   $k^* \leftarrow \min(k_0, \text{BUDGETTOK}(B))$ 
   $routes \leftarrow \text{PLANROUTES}(intent, A, k^*)$ 
  for  $r \in routes$  do
     $C_r \leftarrow \text{EXPANDTOL2}(\text{RETRIEVE}(r, q, S, M, r.budget), M)$ 
  end for
   $F \leftarrow \text{DEDUPLICATE}(\text{WEIGHTEDRRF}(\{C_r\}_{r \in routes}, intent), path, lineSpan)$ 
  if  $\text{WEAKRECALL}(F, S, k^*)$  then
     $F \leftarrow \text{MERGEANDRANK}(F, \text{BACKFILL}(\{C_r\}, k^*), \text{RIPGREG}(q, S, R, k^*))$ 
  end if
   $E \leftarrow \text{PRUNE}(F, B, k^*, scoreGap, duplicateText)$ 
  return  $\text{FILLCONTENT}(E, M)$ 
end function

```

Figure 4. Pseudocode for code search with adaptive top- k retrieval. The query planner controls retrieval depth and route budgets, while the final response remains a compact set of source-grounded L2 snippets.

regions, command status, failure output, and modified files. For edits made through the ContextSniper tool surface, the edit operation validates the target path, applies the source change, and dispatches a memory refresh for the edited file. For long reads and command outputs, the hook writes both the sniped view and the recoverable original into action memory.

The second phase runs at conversation or task boundaries. When a new user prompt arrives, ContextSniper reconciles the indexed state with the working tree, for example by inspecting repository diffs since the previous turn. Changed files are re-chunked or refreshed, deleted snippets are invalidated, and newly relevant regions become available for subsequent searches. This phase catches changes that did not pass through a tool hook and prevents the next repair request from starting from a stale memory image.

The result is a memory hierarchy that is both compact and auditable. Returned snippets cite concrete paths and line regions in the current workspace. Runtime signals cite the command, read, or edit that produced them. Synchronization is therefore not an auxiliary bookkeeping feature; it is the mechanism that makes context sniping trustworthy after the agent starts changing the repository.

3.4 Code Searching with Adaptive Top- k Retrieval

Code search is the main entry point from the host agent into code memory. Given a focused natural-language or keyword query, `search_code` returns a small list of L2 snippets with path and line headers rather than raw grep output. Formally, it computes an evidence packet $E = \{e_1, \dots, e_m\}$ from a query q , scope S , memory hierarchy $M = \{L0, L1, L2\}$, repository state R , requested top- k value k_0 , and context budget B . The adaptive part is the choice of effective return size $m \leq k^*$ and route budgets: traceback-like queries emphasize paths, symbols, and grep terms; behavioral queries allocate more budget to semantic and graph routes; and stateful debugging turns can promote recently touched files from action memory.

The retrieval path is layer aware. A planner assigns the query type and route budgets; seed retrieval searches L0/L1/L2 records; promising L0/L1 nodes expand into L2 leaves; and the final ranker deduplicates by URI, sorts by score, and returns source-grounded L2 evidence. Figure 4 gives the pseudocode.

The hybrid ranker combines semantic embeddings, BM25 lexical documents [31], ctags-style symbol metadata [34], and graph relations over imports, calls, containment, and neighboring-file evidence. ContextSniper fuses route rankings with weighted reciprocal rank fusion by default, or weighted score summation when configured [32]. If indexed retrieval is weak, it falls back to ripgrep-based workspace search [13]. The important contract is that L0 and L1 help find evidence, but the agent sees precise code regions rather than every file or line that matched the query.

3.5 Intention-Aware Filtering and Context Gate

The context gate controls what reaches the model after retrieval or tool execution. For file reads, it preserves relevant local definitions, imports, class or function bodies, and nearby code. For test and shell output, it preserves command lines, exit status, assertion messages, tracebacks, failing paths, and concise success signals. For state, it preserves changed files, touched symbols, repository status, and paths that connect the current turn to earlier evidence. Long

Input: action a , raw output o , intent query q , state R , action memory A , budget B .
Output: prompt-facing view v , sniping marker m , recoverable action-memory record u .

```

function CONTEXTGATE( $a, o, q, R, A, B$ )
   $intent \leftarrow \text{INFERPRESERVATIONINTENT}(a, q, A)$ 
   $meta \leftarrow \text{CLASSIFYOUTPUT}(a, o)$ 
  switch  $meta.type$  do
    case FILEREAD:  $K \leftarrow \text{REQUESTEDREGION}(a, o) \cup \text{LOCALDEFS}(o, intent) \cup \text{NEIGHBORS}(o, intent)$ 
    case COMMANDOUTPUT:  $K \leftarrow \text{COMMANDSTATUS}(a, o) \cup \text{FAILURES}(o) \cup \text{VALIDATION}(o)$ 
    otherwise:  $K \leftarrow \text{STATESIGNALS}(a, o, R, A)$ 
  end switch
   $v \leftarrow \text{RENDERCOMPACTVIEW}(K, B)$ 
   $risk \leftarrow \text{LOWCONFIDENCE}(v, intent) \vee \text{MISSINGCRITICALSIGNAL}(v, meta)$ 
   $u \leftarrow \text{STOREACTIONMEMORY}(L0 = v, L1 = meta, L2 = o)$ 
  if  $\text{SHORTENED}(v, o) \vee risk$  then
     $m \leftarrow \text{SNIPEDMARKER}(u, risk)$ 
     $v \leftarrow \text{ATTACHRECOVERYHINT}(v, m)$ 
  else
     $m \leftarrow \text{NOSNIPING}$ 
  end if
  return  $(v, m, u)$ 
end function

```

Figure 5. Pseudocode for intention-aware filtering and the context gate. The gate keeps action-relevant evidence, records the recoverable original output, and marks shortened views so the host agent can request broader context when needed.

boilerplate, repeated logs, unrelated file regions, and uninformative tails are removed from the prompt-facing view.

The gate is conservative because removing evidence can be worse than spending extra tokens. When a read or command is shortened, ContextSniper stores the original output in action memory and returns a marker indicating that sniping occurred, so the host agent can request broader context if the compact view is insufficient. Figure 5 gives the pseudocode.

3.6 Host-Agent Boundary

ContextSniper is designed as an insertion layer for existing coding agents. The host agent chooses the plan, decides what to edit, and runs validation; the insertion layer controls how repository and runtime evidence are searched, sniped, synchronized, and returned. This boundary allows the mechanism to support a host agent without changing its core reasoning loop. The OpenClaw pilot adapter used in this paper is described in the experimental setup.

4 Evaluation

4.1 Experimental Setup

Table 2. Experimental settings. The embedding model is used only in the ContextSniper condition.

Benchmark	Agent	Base model	Embedding model
SWE-bench Lite	OpenClaw	DeepSeek V4 Flash	text-embedding-3-large
SWE-bench Pro	OpenCode (build)	DeepSeek V4 Flash	text-embedding-3-small

We evaluate ContextSniper on repository-level program repair in two host-agent settings drawn from the SWE-bench benchmark family [3]. Each study uses 50 experimental instances randomly sampled from its benchmark: SWE-bench Lite for OpenClaw and SWE-bench Pro for OpenCode. For each benchmark instance, the runner checks out the target repository at the benchmark-provided base commit, constructs a task prompt from the issue description, and asks the host agent to generate a source-code patch. Generated patches are then scored by the corresponding benchmark validation harness.

The main comparison is each host agent’s baseline versus the ContextSniper pilot implementation. The baseline condition uses the host agent’s default repository tools without the ContextSniper insertion layer. ContextSniper adds local code-memory search, compact evidence assembly, context control for long file-read and command-output results, and source editing through the same memory-aware interface.

We evaluate ContextSniper with OpenClaw, an open-source agent setting where ContextSniper exposes its functions through plugin tools and runtime hooks [14]. The key tool is `search_code`, which is presented as the preferred

replacement for broad grep and exploratory reads; a paired `edit_file` tool validates workspace paths, applies exact source replacements, and refreshes memory for the edited file. Token usage is the primary efficiency metric, including total tokens and, when available, cost and tool/action counts. Validation outcomes are reported cautiously because solve-rate claims require comparable official validation coverage across conditions. The public repository contains the pilot testing scripts and evaluation harnesses used for these experiments, rather than the full AntTrail memory engine.

The OpenCode comparison uses plain OpenCode as the baseline and OpenCode with the full ContextSniper tool bundle as the treatment. Both arms use the `deepseek/deepseek-v4-flash` model and the autonomous build agent, with web tools disabled. The retained re-runs use OpenCode 1.18.12 and a 900-second timeout per instance. Patches are evaluated with the official SWE-bench Pro Docker harness on `linux/amd64`, using the local benchmark image set with image pulling disabled and no host-local scoring fallback. Correctness is the proportion of resolved instances over the 50 experimental instances. Resource metrics are also reported for this 50-instance experiment: token consumption is computed from the `total` field in OpenCode `step_finish` events, and rounds are the number of those events.

The reported OpenCode result is a derived comparison rather than a full re-execution of the treatment arm. It retains the 50 randomly sampled experimental instances and incorporates two accepted ContextSniper re-runs. One replacement both changed the verdict to resolved and reduced tokens and rounds; the other was accepted for its resolved verdict despite using more resources than its historical ContextSniper record. We therefore report the adoption provenance when interpreting the aggregate results.

4.2 Main Results

Across both host-agent settings, the results support an efficiency claim rather than a correctness-improvement claim. On the 50-task OpenClaw SWE-bench Lite comparison, both the OpenClaw baseline and ContextSniper completed 50 task runs and submitted 50 patches. ContextSniper reduces average token use from 1.36M to 0.66M tokens per task, a 51.5% reduction in total tokens. It also reduces logged cost by 36.4% and tool/action count by 46.4% compared with the OpenClaw baseline. Official validation gives comparable submitted-resolution rates, 24.0% for ContextSniper versus 26.0% for the OpenClaw baseline, but the validation-error imbalance prevents a strong repair-success conclusion.

The derived OpenCode comparison shows the same resource-efficiency pattern on the 50 experimental instances randomly sampled from SWE-bench Pro. Plain OpenCode resolves 30 of 50 instances (60.0%), while ContextSniper resolves 29 of 50 (58.0%), a difference of one instance or 2.0 percentage points. Across the 50 experimental instances, ContextSniper reduces total token consumption from 94,173,981 to 56,549,708 tokens (39.95%) and average rounds per instance from 39.98 to 28.73 (28.13%). Thus, ContextSniper retains a small observed correctness deficit while using approximately 37.62 million fewer tokens and substantially fewer interaction rounds. Because the comparison incorporates two accepted re-runs and provides one retained observation per instance, it does not establish statistical or causal superiority.

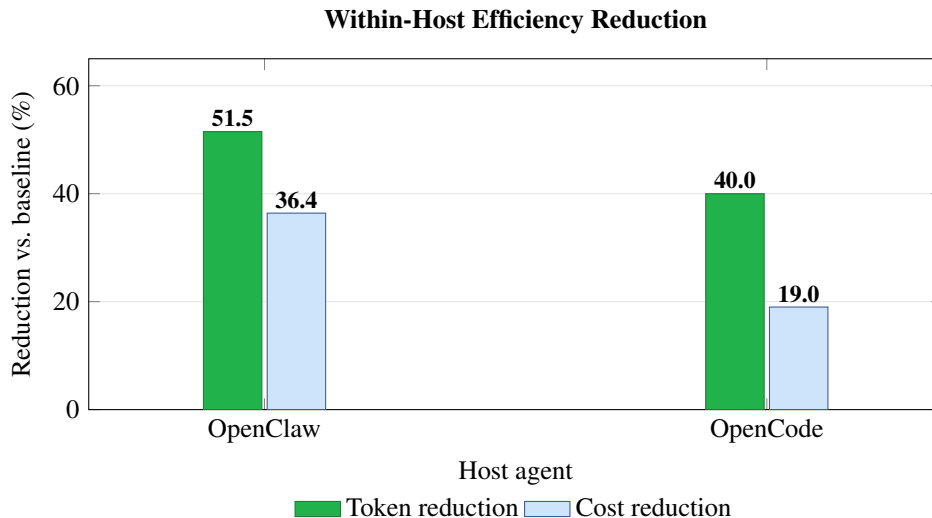


Figure 6. Within-host efficiency reduction relative to each host agent’s baseline setting. Absolute token and cost values are reported in Table 3.

Table 3 reports the metrics shared by the two 50-instance experiments while preserving host-specific measurement

Table 3. Main efficiency and validation results for OpenClaw on SWE-bench Lite and OpenCode on SWE-bench Pro. Each method completed 50 experimental runs. ‘Val. OK’ reports validation runs that completed without validator error, not agent task completion. ‘Avg. Int.’ is the host-native interaction measure: actions for OpenClaw and `step_finish` rounds for OpenCode; values are comparable only within a host. ‘Ctx. Act.’ counts context-acquisition actions and is available only for OpenClaw. Dashes denote metrics not reported by the corresponding experiment.

Benchmark	Host	Method	Runs	Patch	Val. OK	Res.	Res. Rate	Total Tok.	Cost	Avg. Int.	Ctx. Act.	Rel. Tok.
Lite	OpenClaw	Baseline	50	50	26	13	26.0%	67.80M	\$0.635	32.1	572	–
Lite	OpenClaw	ContextSniper	50	50	33	12	24.0%	32.91M	\$0.404	17.2	461	-51.5%
Lite	OpenClaw	Rel. change	–	–	+7	-1	-2.0 pp	-51.5%	-36.4%	-46.4%	-19.4%	–
Pro	OpenCode	Baseline	50	50	50	30	60.0%	94.17M	\$0.0147	39.98	–	–
Pro	OpenCode	ContextSniper	50	50	50	29	58.0%	56.55M	\$0.0119	28.73	–	-40.0%
Pro	OpenCode	Rel. change	–	–	0	-1	-2.0 pp	-40.0%	-19.0%	-28.1%	–	–

definitions. The OpenCode experiment reports 50 submitted patches in both conditions and logged costs of \$0.0147 for the baseline and \$0.0119 for ContextSniper, a 19.0% reduction. It does not provide a context-acquisition action count, so those cells remain unreported. A five-task Django pilot with additional retrieval and memory systems is useful as exploratory evidence, but it is not used as a main competitor claim because it does not yet match the 50-task protocol.

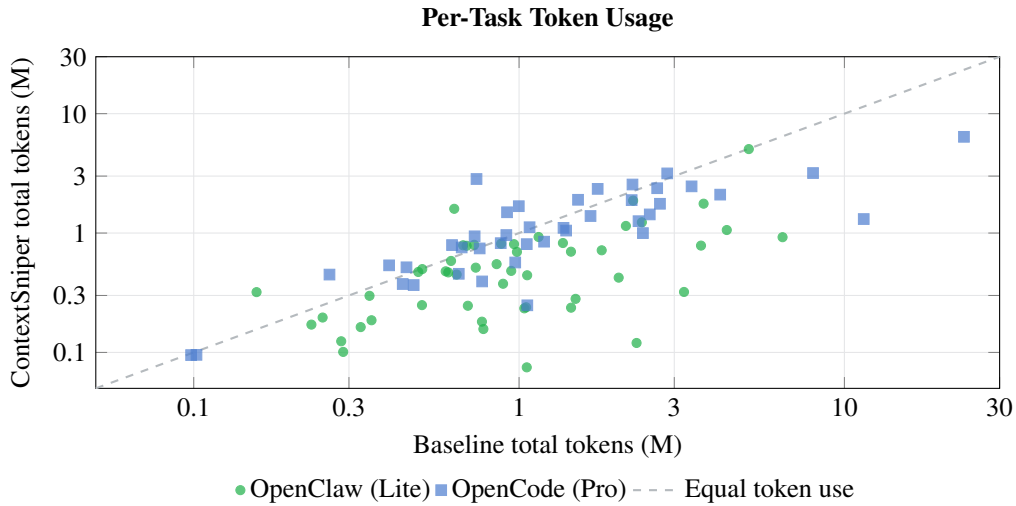


Figure 7. Per-task token-usage comparison on matched benchmark tasks: 50 SWE-bench Lite tasks for OpenClaw and 41 SWE-bench Pro tasks for OpenCode. Each point compares the host-agent baseline and ContextSniper for one task. Both axes use a logarithmic scale; points below the diagonal indicate tasks where ContextSniper uses fewer tokens.

4.3 Task-Level Failure Analysis

The matched OpenClaw comparison shows the intended behavior on most tasks: ContextSniper uses fewer tokens on 45 of 50 matched tasks. The remaining 5 cases are more informative than the aggregate average because they show when a context-access layer can add overhead rather than remove it. Trace inspection separates these increases into two patterns.

First, some increases are dominated by fixed retrieval and tool-interface overhead. On `pylint-dev__pylint-7228`, `pallets__flask-5063`, and `matplotlib__matplotlib-24265`, ContextSniper uses the same number of total actions as the OpenClaw baseline, but adds two to four code-search calls. The resulting token increases are modest, between 9.6% and 17.3%. These are not cases where the agent spirals into much longer exploration; rather, the task is short enough that the retrieval layer’s setup and search turns are not fully amortized by shorter later context.

Second, larger increases occur when retrieval does not quickly end exploration or when the baseline run is unusually short. The clearest retrieval-overhead failure is `mwaskom__seaborn-3407`: ContextSniper uses six code-search calls and 32 actions, compared with 21 baseline actions, raising token use by 152.3%. This suggests a retrieval miss or insufficiently decisive evidence packet that caused the host agent to continue searching. In `pytest-dev__pytest-7168`, ContextSniper retrieves the relevant `saferrepr.py` region early, but the baseline completes the task in only five actions; even after rerun, the ContextSniper path uses nine actions and two search calls, making the relative token increase 104.6%. This is an overhead-on-short-task case rather than a pure localization failure. Separately, `pytest-dev__pytest-7220` was a large pre-rerun increase, but after rerun it becomes a token reduction; the accepted trace still records backend

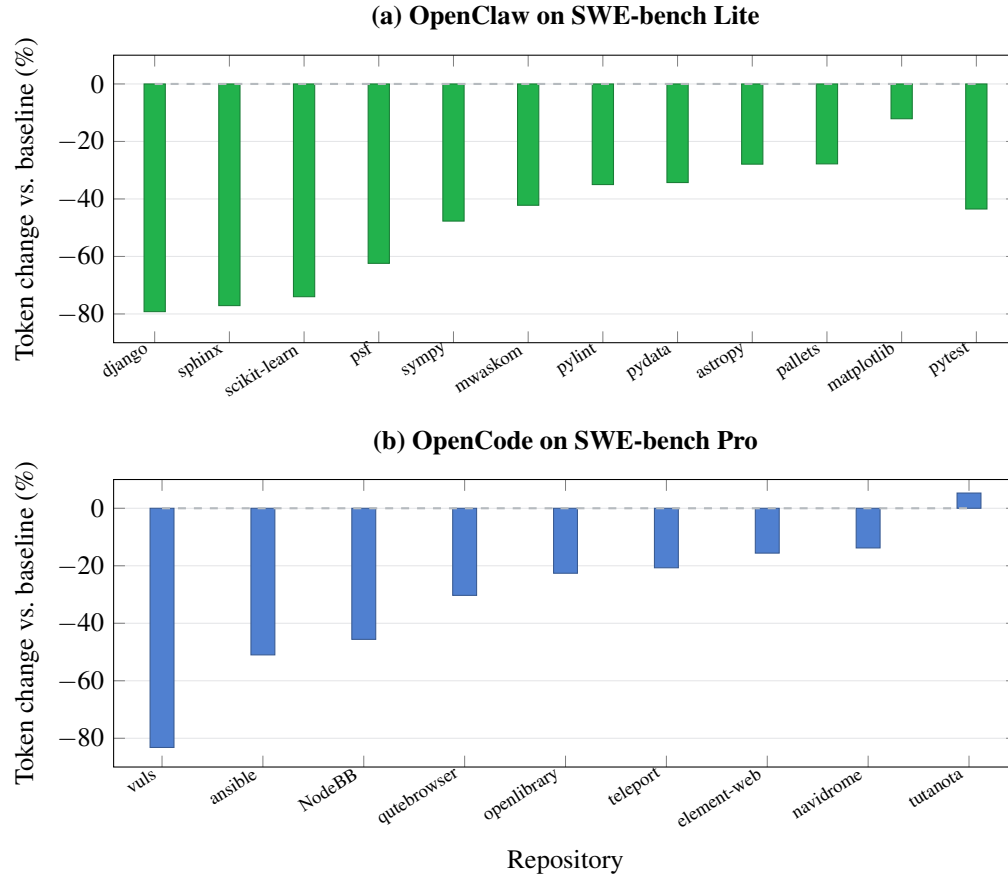


Figure 8. Repository-level token change on the matched benchmark task sets for (a) OpenClaw on SWE-bench Lite and (b) OpenCode on SWE-bench Pro. Values aggregate total tokens by repository before computing relative change against each host-agent baseline. Negative values indicate token savings from ContextSniper. Repositories in panel (b) are ordered from the largest token reduction to the positive token increase.

startup warnings, so we exclude it from the current token-increase set.

Figure 9 complements this task-level analysis by measuring whether the retriever surfaces the eventual target file within the first returned paths. Low-recall repositories identify where the first failure pattern is more likely: if the target file is absent from the early results, the host agent must either issue more ContextSniper searches or fall back to broader native exploration. The token-increase cases therefore do not overturn the efficiency result, but they calibrate it: ContextSniper is strongest when early retrieval replaces broad exploration, and weakest when retrieval overhead is added to an already short baseline run or when the first evidence packet misses the decisive file.

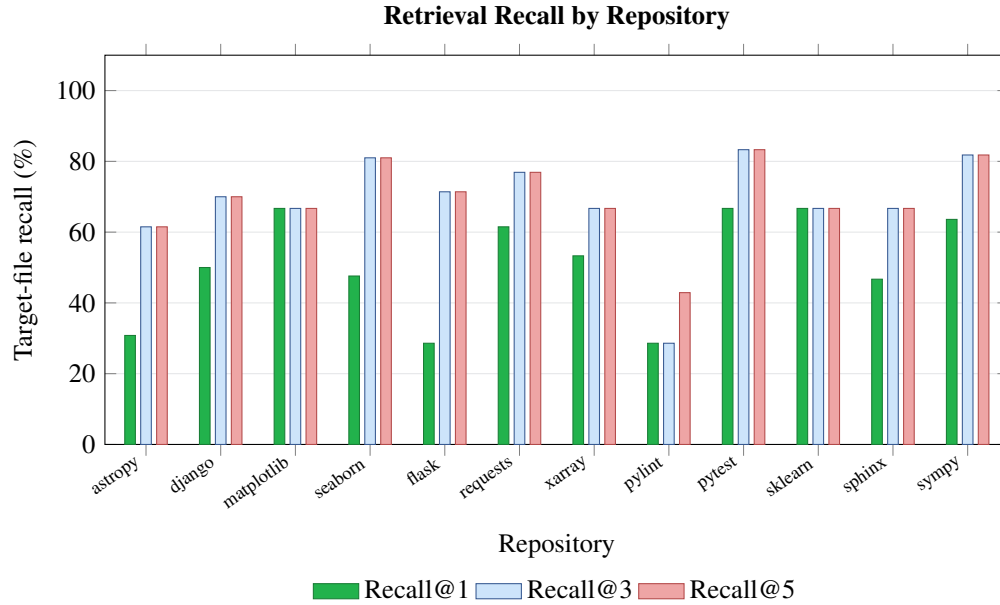


Figure 9. Retrieval recall by repository from actual ContextSniper search samples labeled with benchmark patch target files. The denominator is search attempts per repository; Recall@k reports whether a target file appears in the top-k returned paths.

4.4 Comparison with Memory and RAG Systems

To contextualize ContextSniper against related memory-style integrations, we run a five-task Django pilot using the same OpenClaw host setting. The pilot compares the OpenClaw baseline, ContextSniper, and several memory or retrieval integrations, including mem0, Letta/MemGPT, OpenViking, TencentDB Memory, Serena, and a LlamaIndex-style RAG baseline [35–40]. All methods complete five task runs and are evaluated with official SWE-bench validation reports.

Figure 10 should be read as exploratory evidence rather than the main benchmark because its five-task size is much smaller than the matched OpenClaw comparison in Table 3. Task outcomes are similar: all methods resolve four of five pilot tasks except OpenViking, which resolves five of five. We therefore use the figure to compare token-efficiency patterns, not to make a final solve-rate claim.

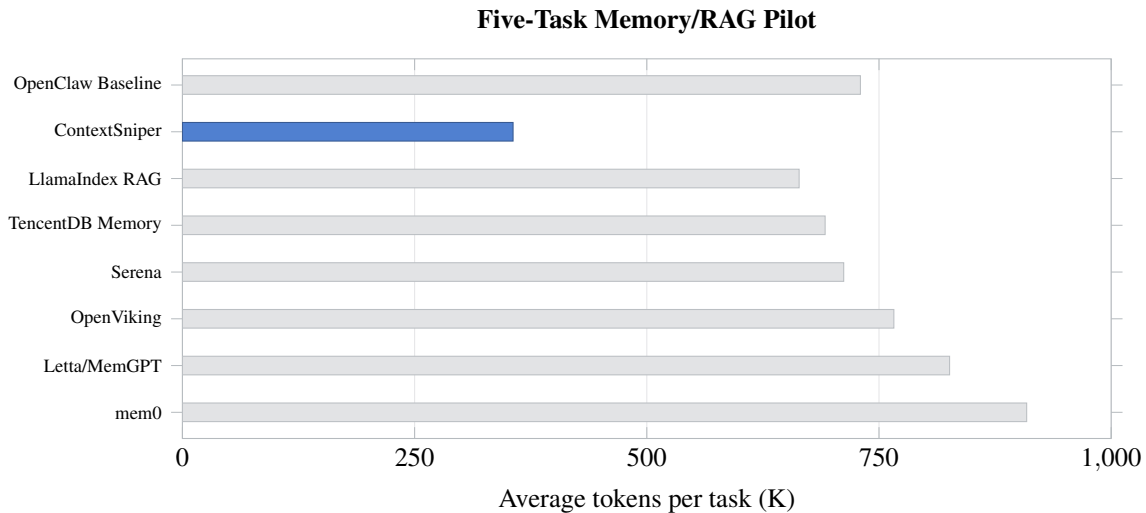


Figure 10. Five-task Django pilot comparison with memory and RAG-style integrations. Bar length shows average logged tokens per task. The pilot is exploratory and smaller than the main 50-task OpenClaw comparison; task-resolution counts are reported in the text.

5 Discussion

5.1 ContextSniper as AntTrail Infrastructure

ContextSniper treats code memory and context control as infrastructure for coding agents. Its central design choice is to retrieve broadly, expose narrowly, and preserve recoverable context outside the prompt. Placing this memory layer outside the host agent’s reasoning loop lets the same substrate support different tool interfaces, as demonstrated by the OpenClaw pilot adapter.

This positioning also clarifies the role of the “sniper” metaphor. ContextSniper is not simply a larger index or a summarizer for entire repositories. Its job is to remove unwanted context before the LLM spends attention on it: irrelevant file regions, redundant exploration, noisy command output, stale snippets, and broad repository dumps. The system is useful when it keeps the concrete evidence that matters for a repair while preventing surrounding noise from filling the context window.

The result is a division of labor between agent and infrastructure. The host agent decides what problem to solve and what patch to write. ContextSniper decides how to expose repository evidence under a limited context budget. This makes context management an explicit systems layer rather than an implicit side effect of file reads and shell commands.

5.2 Deployment

ContextSniper can be deployed through host-agent plugins because its boundary is intentionally narrow: it mediates context access while leaving planning, editing, and validation with the host agent. The pilot adapters in this paper instantiate that boundary for two tool surfaces, but the released repository is best understood as the testing and evaluation harness around the specialization rather than as the full memory engine.

The AntTrail framing matters for scope. This paper evaluates the repository-level repair setting, while the parent memory engine targets broader cross-domain retrieval and context management. The results therefore characterize the coding specialization, not the complete AntTrail system.

5.3 Practical Implications

The most direct implication is lower token cost. If the agent receives fewer irrelevant file regions and shorter command outputs, it pays for less input context and often needs fewer exploratory turns. This matters for repository-level repair because the expensive part of many runs is not only patch generation, but the repeated process of finding the files, symbols, and tests that matter.

ContextSniper also makes the agent’s working context more predictable. Raw file reads and shell commands can vary widely in size, and a single broad output can dominate a turn. Context control gives the system a place to enforce budgets and remove output that is unlikely to help. The same logs can make debugging easier: developers can inspect which snippets were retrieved, which outputs were suppressed, and whether the final patch touched files that ContextSniper surfaced early.

For local-codebase workflows, this design preserves the agent’s normal interaction style. Developers do not need to manually curate a prompt for each issue, and the agent does not need to ingest an entire repository. Instead, ContextSniper provides a local memory layer that can be queried and refreshed as the repository changes. This is especially useful for private or evolving codebases where relevant evidence is spread across implementation files, tests, configuration, and runtime output.

5.4 Limitations and Future Work

Broader conclusions about repair success require more standardized validation, more tasks, and more repositories. A SWE-bench Verified evaluation would strengthen the evidence, and future work should report confidence intervals so token reductions can be separated from benchmark and validation noise. The present results should also not be read as an evaluation of the full cross-domain AntTrail engine.

ContextSniper also depends on retrieval and indexing quality. Embedding models, chunking strategies, symbol extraction, and structural links all affect which snippets are surfaced. If the relevant evidence is weakly signaled, spread across unusual files, or visible only after running tests, retrieval may miss it. Better hybrid retrieval, subtask-aware recall, and failure-aware reranking are natural extensions.

Context control introduces its own risk. The system can remove useful evidence if the sniping policy is too aggressive or if the task requires broad architectural context. Agents may need an explicit way to request broader output, expand a shortened region, or mark a file as important. Future work should study learned sniping policies, budget sweeps, and human-inspectable traces that explain why particular snippets were kept or removed. Commit-history memory is

another promising direction because prior changes and linked issues may reveal repair-relevant files that current-source retrieval alone misses.

6 Conclusion

ContextSniper makes repository code-context access more token-efficient by removing unwanted context and preserving repair-relevant evidence. The disclosed OpenClaw results show substantial reductions in context cost while keeping the host agent’s normal workflow. More broadly, the paper argues that coding agents should treat code memory and context control as explicit infrastructure, not as incidental byproducts of raw file reads and shell commands.

References

- [1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [2] M. Vogel, F. Meyer-Eschenbach, S. Kohler, E. Grünwald, and F. Balzer, “Codebase-memory: Tree-sitter-based knowledge graphs for LLM code exploration via MCP,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.27277>
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [4] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.05427>
- [5] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based software engineering agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.01489>
- [6] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.00615>
- [7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: <https://arxiv.org/abs/2307.03172>
- [8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [9] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, “RepoGraph: Enhancing AI software engineering with repository-level code graph,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.14684>
- [10] DataGallery, “AntTrail: Agent memory engine,” 2026. [Online]. Available: <https://gitcode.com/datagallery/AntTrail>
- [11] B. Yang, J. Ren, S. Jin, Y. Liu, F. Liu, B. Le, and H. Tian, “Enhancing repository-level software repair via repository-aware knowledge graphs,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.21710>
- [12] B. Wang, W. Xu, Y. Li, M. Gao, Y. Xie, H. Sun, and D. Chen, “Improving code localization with repository memory,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.01003>
- [13] A. Gallant, “ripgrep.” [Online]. Available: <https://github.com/BurntSushi/ripgrep>
- [14] OpenClaw Contributors, “OpenClaw: Personal AI assistant.” [Online]. Available: <https://github.com/openclaw/openclaw>
- [15] S. Chen, S. Lin, Y. Shi, H. Lian, X. Gu, L. Yun, D. Chen, L. Cao, J. Liu, N. Xia, and Q. Wang, “SWE-Exp: Experience-driven software issue resolution,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.23361>
- [16] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, “EXPEREPAIR: Dual-memory enhanced LLM-based repository-level program repair,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.10484>
- [17] Q. Wang, Z. Cheng, S. Zhang, F. Liu, R. Xu, H. Lian, K. Wang, X. Yu, J. Yin, S. Hu, Y. Hu, S. Zhang, Y. Liu, R. Chen, and H. Wang, “MemGovern: Enhancing code agents through learning from governed human experiences,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.06789>

- [18] K. Shen, J. Zhang, C. Sun, W. Zeng, and Y. Yue, “Structurally aligned subtask-level memory for software engineering agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.21611>
- [19] M. Li, T. Chen, G. Yang, and J. Li, “MEMCoder: Multi-dimensional evolving memory for private-library-oriented code generation,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.24222>
- [20] M. Li, L. H. Xu, Q. Tan, T. Cao, and Y. Liu, “Learning to commit: Generating organic pull requests via online repository memory,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.26664>
- [21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.05736>
- [22] —, “LongLLMLingua: Extending LLMs’ context windows without tuning,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.12957>
- [23] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.04408>
- [24] C. Yoon, T. Lee, H. Hwang, M. Jeong, and J. Kang, “COMPACT: Compressing retrieved documents actively for question answering,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.09014>
- [25] Selective Context Contributors, “Selective Context: Compress input to ChatGPT or other LLMs,” 2023. [Online]. Available: https://github.com/liyucheng09/Selective_Context
- [26] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, “Compressing context to enhance inference efficiency of large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 6322–6333. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.391/>
- [27] ctxbudgeter Contributors, “ctxbudgeter: ContextOps toolkit for production AI agents.” [Online]. Available: <https://github.com/Kayariyan28/ctxbudgeter>
- [28] RTK Contributors, “RTK: Rust token killer.” [Online]. Available: <https://github.com/rtk-ai/rtk>
- [29] Headroom Contributors, “Headroom: The context compression layer for AI agents.” [Online]. Available: <https://github.com/headroomlabs-ai/headroom>
- [30] Bearing Contributors, “Bearing: Task runner for directing AI coding agents.” [Online]. Available: <https://github.com/rocketvish/bearing>
- [31] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” in *Foundations and Trends in Information Retrieval*, 2009, vol. 3, no. 4, pp. 333–389.
- [32] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [33] AGFS Contributors, “AGFS: Agent file system.” [Online]. Available: <https://github.com/c4pt0r/agfs>
- [34] Universal Ctags Contributors, “Universal ctags.” [Online]. Available: <https://github.com/universal-ctags/ctags>
- [35] mem0 Contributors, “mem0: Universal memory layer for AI agents.” [Online]. Available: <https://github.com/mem0ai/mem0>
- [36] Letta Contributors, “Letta: Stateful agents and MemGPT.” [Online]. Available: <https://github.com/letta-ai/letta>
- [37] OpenViking Contributors, “OpenViking: Context database for AI agents.” [Online]. Available: <https://github.com/volcengine/OpenViking>
- [38] TencentDB Agent Memory Contributors, “TencentDB Agent Memory.” [Online]. Available: <https://github.com/TencentCloud/TencentDB-Agent-Memory>
- [39] Serena Contributors, “Serena: Semantic coding toolkit for agents.” [Online]. Available: <https://github.com/oraios/serena>
- [40] LlamaIndex Contributors, “LlamaIndex: Framework for agentic applications and retrieval-augmented generation.” [Online]. Available: https://github.com/run-llama/llama_index